Lecture 06: Binary Heap Mechanics

Topics

- Binary heap as an array-based complete binary tree
- Parent/child index arithmetic
- Heapify-up (sift-up) on insert
- Heapify-down (sift-down) on pop
- Build-heap in O(n) vs naïve O(n log n)
- Why heaps are cache-friendly

Goals

- Understand *why* `heapq` runs in O(log n) — not just that it does
- Implement a `MinHeap` class from scratch
- Visualize every step of insert, pop, and build-heap
- Explain the O(n) build-heap surprise and where it comes from

Roadmap

First half (≈45 min)

- From black box to glass box: what's inside `heapq`?
- Complete binary trees and the array layout
- Index arithmetic: parent, left child, right child
- Heap property and the min-heap invariant
- Insert: heapify-up step by step
- Pop: heapify-down step by step
- In-class exercise 1 (commit required)

Break (3 min)

Second half (≈45 min)

- `MinHeap` class: full implementation
- Build-heap: naïve vs Floyd's O(n) algorithm
- Why the O(n) result is surprising (and correct)
- Cache-friendliness: why array layout matters
- In-class exercise 2 (commit required)
- Complexity table and wrap-up

From black box to glass box

Last lecture, we used `heapq` like a vending machine:

```
heapq.heappush(pq, (priority, item))   # O(log n)
heapq.heappop(pq)                       # O(log n)
pq[0]                                   # O(1)
```

Today we open the machine. We'll answer:

- Why does `heappush` take O(log n), not O(n)?
- Why is the minimum *always* at index 0?
- How does `heapify` build a valid heap from a random list in O(n)?
- Why does a plain list outperform a tree of linked nodes?

Understanding the internals makes you a better user — and a better interviewer.

Binary heap: the key idea

A **binary heap** is a data structure that satisfies two properties simultaneously:

1. Shape property: complete binary tree

Every level is fully filled except possibly the last, which fills **left to right**.

```
        1          ← level 0 (root)
      /   \
     3     2       ← level 1
    / \   /
   7   5 4         ← level 2 (last level, fills left to right)
```

2. Heap property: min-heap ordering

Every node is ≤ both its children. The minimum is always at the **root**.

```
Parent ≤ left child   AND   Parent ≤ right child
```

These two properties together give us O(log n) insert and pop.

The key insight: store it as an array

Because the tree is *complete*, we can store it in a plain list — **no pointers needed**.

```
Tree:                  Array:
         1             index:  0  1  2  3  4  5
       /   \           value: [1, 3, 2, 7, 5, 4]
      3     2
     / \   /
    7   5 4
```

Reading level by level, left to right → fills the array perfectly.

Given index `i`: | Relationship | Formula | |---|---| | Left child | `2*i + 1` | | Right child | `2*i + 2` | | Parent | `(i - 1) // 2` |

No `Node` class. No `.left` / `.right` pointers. Just arithmetic.

Index arithmetic — live demo

In [ ]:

```python
# The three formulas that run the entire heap

def left(i):   return 2 * i + 1
def right(i):  return 2 * i + 2
def parent(i): return (i - 1) // 2

heap = [1, 3, 2, 7, 5, 4]

print(f"Array: {heap}")
print()
print(f"Root (index 0): {heap[0]}")
print()

for i in range(len(heap)):
    lc = left(i)
    rc = right(i)
    pa = parent(i)
    lv = heap[lc] if lc < len(heap) else "—"
    rv = heap[rc] if rc < len(heap) else "—"
    pv = heap[pa] if i > 0 else "(root)"
    print(f"  index {i} (val={heap[i]}): "
          f"parent={pv!s:<8} "
          f"left={lv!s:<6} "
          f"right={rv!s:<6}")
```

Visualizing the heap as a tree

We'll use this helper throughout the lecture to render a heap array as a tree.

In [ ]:

```
def print_heap(heap, label="", highlight=None):
    """Print a heap array as an indented tree.

    highlight: set of indices to mark with ★
    """
    if label:
        print(f"\n{'─'*40}")
        print(f"  {label}")
        print(f"{'─'*40}")
    if not heap:
        print("  (empty)")
        return

    highlight = highlight or set()
    n = len(heap)

    def _display(i, prefix="", is_left=True):
        if i >= n:
            return
        connector = "├── " if is_left else "└── "
        marker = " ★" if i in highlight else ""
        if i == 0:
            print(f"  {heap[i]}{marker}")
        else:
            print(f"  {prefix}{connector}{heap[i]}{marker}")
        child_prefix = prefix + ("│   " if is_left else "    ")
        l, r = 2*i+1, 2*i+2
        if l < n:
            _display(l, child_prefix, is_left=True)
        if r < n:
```

```
            _display(r, child_prefix, is_left=False)

    _display(0)
    print(f"  Array: {heap}")

print_heap([1, 3, 2, 7, 5, 4], label="Starting heap")
```

Insert: heapify-up (sift-up)

Algorithm:

1. Append the new element at the end of the array (next open position in the last level)
2. Compare it with its **parent**
3. If the new element is smaller than its parent → **swap**
4. Repeat from step 2 until the element is in the right place

```
Insert 0 into [1, 3, 2, 7, 5, 4]:

Step 0: append     [1, 3, 2, 7, 5, 4, 0]   ← 0 is at index 6
Step 1: 0 < parent(6) = heap[2] = 2 → swap  [1, 3, 0, 7, 5, 4, 2]   ← now at index 2
Step 2: 0 < parent(2) = heap[0] = 1 → swap  [0, 3, 1, 7, 5, 4, 2]   ← now at index 0 (root!)
Step 3: at root → stop
```

Cost: at most O(log n) swaps — one per level of the tree.

Insert: step-by-step visualization

In [ ]:

```
def heap_insert_trace(heap, val):
    """Insert val into heap and print every step."""
    heap = heap[:]
    heap.append(val)
    i = len(heap) - 1

    print_heap(heap, label=f"Step 0: appended {val} at index {i}", high

    step = 1
    while i > 0:
        p = (i - 1) // 2
        if heap[i] < heap[p]:
            heap[i], heap[p] = heap[p], heap[i]
            print_heap(
                heap,
                label=f"Step {step}: swapped index {i}↔{p} "
                      f"(now {heap[p]} up, {heap[i]} down)",
                highlight={p},
            )
            i = p
            step += 1
        else:
            print(f"\n  Step {step}: heap[{i}]={heap[i]} ≥ parent heap[
            break
    else:
        print(f"\n  Reached root → done")

    return heap

start = [1, 3, 2, 7, 5, 4]
```

```
print("Inserting 0 into:", start)
result = heap_insert_trace(start, 0)
print(f"\nFinal heap: {result}")
```

Pop: heapify-down (sift-down)

Algorithm:

1. The minimum is always at index 0 — save it to return
2. Move the **last element** to the root (index 0) and shrink the array
3. Compare the root with its **smaller child**
4. If the root is larger than its smaller child → **swap**
5. Repeat from step 3 until the element reaches a valid position

```
Pop from [1, 3, 2, 7, 5, 4]:

Step 0: remove root (1), move last (4) to root
         [4, 3, 2, 7, 5]        ← 4 is at index 0
Step 1: children of 0 are heap[1]=3, heap[2]=2; smaller child = 2
         4 > 2 → swap           [2, 3, 4, 7, 5]
Step 2: 4 is at index 2; children are heap[5]=—, heap[6]=—; no children → done
```

Cost: at most O(log n) swaps — one per level of the tree.

Pop: step-by-step visualization

In [ ]:

```
def heap_pop_trace(heap):
    """Pop the minimum from heap and print every step."""
    heap = heap[:]
    if not heap:
        return None, []

    min_val = heap[0]
    last = heap.pop()   # remove last element

    if not heap:
        return min_val, []

    heap[0] = last      # put it at the root
    i = 0
    n = len(heap)

    print_heap(heap, label=f"Step 0: moved last ({last}) to root, remove
               highlight={0})

    step = 1
    while True:
        l, r = 2*i+1, 2*i+2
        smallest = i

        if l < n and heap[l] < heap[smallest]:
            smallest = l
        if r < n and heap[r] < heap[smallest]:
            smallest = r

        if smallest == i:
```

```
            print(f"\n  Step {step}: index {i} is ≤ both children → done
            break

        heap[i], heap[smallest] = heap[smallest], heap[i]
        print_heap(
            heap,
            label=f"Step {step}: swapped index {i}↔{smallest} "
                  f"({heap[i]}↑  {heap[smallest]}↓)",
            highlight={smallest},
        )
        i = smallest
        step += 1

    return min_val, heap

start = [1, 3, 2, 7, 5, 4]
print_heap(start, label="Starting heap")
min_val, result = heap_pop_trace(start)
print(f"\nPopped: {min_val}")
print(f"Final heap: {result}")
```

Why log n? The tree height argument

A complete binary tree with **n nodes** has height $\lfloor \log_2 n \rfloor$.

n	height	max swaps
7	2	2
15	3	3
1,000	9	9
1,000,000	19	19
1,000,000,000	29	29

Both insert (heapify-up) and pop (heapify-down) travel at most one full root-to-leaf path.

That path has length $\log_2 n$ — so both operations are O(log n).

One billion elements → at most 30 comparisons. That's why heaps are used everywhere.

Heap property: the invariant

At any point, a valid min-heap satisfies:

```
For every index i > 0:  heap[(i-1)//2]  ≤  heap[i]
```

In words: **every parent is ≤ both its children.**

This does NOT mean the array is fully sorted:

```
heap = [1, 3, 2, 7, 5, 4]

  index 0 (val=1): parent of 1 (3) and 2 (2)  → 1 ≤ 3 ✓   1 ≤ 2 ✓
  index 1 (val=3): parent of 7 and 5           → 3 ≤ 7 ✓   3 ≤ 5 ✓
  index 2 (val=2): parent of 4 (only child)    → 2 ≤ 4 ✓
  indices 3–5: leaves, no children
```

Notice `heap[1]=3 > heap[2]=2` — siblings are **not** ordered relative to each other. Only the parent-child relationship is enforced.

In [ ]:

```python
def is_min_heap(heap):
    """Verify the min-heap property holds for every node."""
    n = len(heap)
    violations = []
    for i in range(1, n):
        p = (i - 1) // 2
        if heap[p] > heap[i]:
            violations.append((p, i, heap[p], heap[i]))
    if violations:
        for p, i, pv, iv in violations:
            print(f"  VIOLATION: heap[{p}]={pv} > heap[{i}]={iv}")
        return False
    print("  ✓ Valid min-heap")
    return True

print("[1, 3, 2, 7, 5, 4]:")
is_min_heap([1, 3, 2, 7, 5, 4])   # valid

print()
print("[1, 3, 2, 7, 5, 0]:")
is_min_heap([1, 3, 2, 7, 5, 0])   # invalid: 2 > 0

print()
print("[2, 3, 1, 7, 5, 4]:")
is_min_heap([2, 3, 1, 7, 5, 4])   # invalid: 2 > 1 (right child)
```

In-class Exercise 1 (commit required)

Topic: Heap property and heapify-up

1. **Multiple choice:** After inserting a new element at the end of a min-heap, heapify-up compares the new element with:

    - A. Its left child
    - B. Its right child
    - C. Its parent
    - D. The root

2. **Short answer (2–3 sentences):** A complete binary tree with 100 elements has how many levels? Why does that bound the cost of insert and pop?

3. **Coding task:** Implement `sift_up(heap, i)` — the heapify-up operation — **without** using the `heap_insert_trace` function above.

    - Input: a list `heap` and starting index `i`
    - Mutates `heap` in-place (no return value needed)
    - Test by inserting values one at a time and checking `is_min_heap` after each insert
    - Bonus: modify to trace which indices are being compared at each step

4. **Commit:**

```
git add lecture/Lecture06_Binary_Heap_Mechanics.ipynb
git commit -m "Lecture 06 exercise 1 work"
```

In [ ]:

```python
# Exercise 1 workspace

def sift_up(heap, i):
    """Restore heap property by bubbling heap[i] upward.

    Mutates heap in-place.
    """
    # YOUR CODE HERE
    pass

# Test: insert elements one at a time and verify the heap property holds
heap = []
for val in [5, 3, 8, 1, 9, 2, 7]:
    heap.append(val)
    sift_up(heap, len(heap) - 1)
    print(f"After inserting {val}: {heap}")
    is_min_heap(heap)
    print()
```

In [ ]:
```
# —— SOLUTION HIDDEN ——
# Complete this after the exercise prompt above.
```

Break (3 minutes)

- Stand up, stretch, reset.
- When you come back: we build the full `MinHeap` class and discover the O(n) build-heap trick.

MinHeap class: putting it all together

Now that we understand sift-up and sift-down, let's build a complete `MinHeap` from scratch.

API we'll implement

Method	Behavior	Cost
`push(val)`	Insert a value	O(log n)
`pop()`	Remove and return minimum	O(log n)
`peek()`	View minimum without removing	O(1)
`__len__()`	Number of elements	O(1)
`build(iterable)`	Initialize from any iterable	O(n)

No external imports. No `heapq`. Pure Python.

In [ ]:

```
class MinHeap:
    """A min-heap backed by a Python list.

    All operations use only index arithmetic — no pointers, no Node obje
    """

    def __init__(self):
        self._data = []

    # ── Index helpers ──────────────────────────────────────────────────

    @staticmethod
    def _parent(i):      return (i - 1) // 2

    @staticmethod
    def _left(i):        return 2 * i + 1

    @staticmethod
    def _right(i):       return 2 * i + 2

    # ── Core sift operations ──────────────────────────────────────────

    def _sift_up(self, i):
        """Bubble element at i upward until heap property is restored."""
        data = self._data
        while i > 0:
            p = self._parent(i)
            if data[i] < data[p]:
                data[i], data[p] = data[p], data[i]
                i = p
```

```
            else:
                break

    def _sift_down(self, i):
        """Push element at i downward until heap property is restored."""
        data = self._data
        n = len(data)
        while True:
            smallest = i
            l = self._left(i)
            r = self._right(i)
            if l < n and data[l] < data[smallest]:
                smallest = l
            if r < n and data[r] < data[smallest]:
                smallest = r
            if smallest == i:
                break
            data[i], data[smallest] = data[smallest], data[i]
            i = smallest

    # ── Public API ──────────────────────────────────────────────

    def push(self, val):
        """Insert val in O(log n)."""
        self._data.append(val)
        self._sift_up(len(self._data) - 1)

    def pop(self):
        """Remove and return the minimum in O(log n)."""
        if not self._data:
            raise IndexError("pop from empty heap")
```

```
        data = self._data
        # Swap root with last element, then shrink
        data[0], data[-1] = data[-1], data[0]
        val = data.pop()
        if data:
            self._sift_down(0)
        return val

    def peek(self):
        """Return (don't remove) the minimum in O(1)."""
        if not self._data:
            raise IndexError("peek at empty heap")
        return self._data[0]

    def __len__(self):
        return len(self._data)

    def __repr__(self):
        return f"MinHeap({self._data})"
```

In [ ]:

```python
# Quick smoke test against heapq
import heapq

values = [5, 3, 8, 1, 9, 2, 7, 4, 6]

# Our MinHeap
mine = MinHeap()
for v in values:
    mine.push(v)
my_order = [mine.pop() for _ in range(len(mine) + len(values))  # drain
            if len(mine) > 0]

# heapq reference
ref = []
for v in values:
    heapq.heappush(ref, v)
ref_order = [heapq.heappop(ref) for _ in range(len(values))]

print("Our MinHeap output:", my_order)
print("heapq output:      ", ref_order)
print("Match:", my_order == ref_order)
```

In [ ]:

```python
# Cleaner test
import heapq

values = [5, 3, 8, 1, 9, 2, 7, 4, 6]

mine = MinHeap()
for v in values:
    mine.push(v)

ref = values[:]
heapq.heapify(ref)

my_order  = []
ref_order = []
while len(mine) > 0:
    my_order.append(mine.pop())
while ref:
    ref_order.append(heapq.heappop(ref))

print("Our MinHeap output:", my_order)
print("heapq output:      ", ref_order)
print("Match:", my_order == ref_order)
```

Build-heap: two approaches

Given an unsorted list, how do we turn it into a valid heap?

Approach 1: insert one at a time — O(n log n)

```
heap = MinHeap()
for val in data:          # n iterations
    heap.push(val)        # each push: O(log n)
# Total: O(n log n)
```

Approach 2: Floyd's algorithm — O(n) ← the surprise!

1. Copy all elements into an array (no ordering)
2. Start from the **last non-leaf** and sift-down each node toward the root

```
data = list(iterable)                          # O(n)
n = len(data)
for i in range((n - 2) // 2, -1, -1):          # from last non-leaf to
root
    sift_down(data, i)                         # O(log n) per node...
but?
```

Naively this looks like O(n log n) — but it's actually **O(n)**. Why?

Why Floyd's build-heap is O(n)

Key insight: **most nodes are near the bottom of the tree** and have very short sift-down paths.

```
In a complete binary tree with n nodes:
  ≈ n/2   nodes are leaves          → 0 swaps each
  ≈ n/4   nodes are at height 1     → ≤ 1 swap each
  ≈ n/8   nodes are at height 2     → ≤ 2 swaps each
  ...     ...
  1 node  is the root (height log n) → ≤ log n swaps
```

Total work = sum over all heights:

```
∑ (n / 2^(h+1)) * h   for h = 0 to log n
= n * ∑ h / 2^(h+1)
= n * 1                (geometric series)
= O(n)
```

The leaves do nothing. The work concentrates at the root — and there's only one root. The total is bounded by n × (a constant), hence **O(n)**.

Build-heap: Floyd's algorithm — step by step

In [ ]:

```
def build_heap_trace(data):
    """Build a min-heap in-place using Floyd's algorithm, printing every
    heap = data[:]
    n = len(heap)
    last_non_leaf = (n - 2) // 2

    print(f"Starting array: {heap}")
    print(f"Last non-leaf index: {last_non_leaf} (value={heap[last_non_

    for i in range(last_non_leaf, -1, -1):
        before = heap[:]
        # sift_down in-place
        j = i
        while True:
            smallest = j
            l, r = 2*j+1, 2*j+2
            if l < n and heap[l] < heap[smallest]: smallest = l
            if r < n and heap[r] < heap[smallest]: smallest = r
            if smallest == j: break
            heap[j], heap[smallest] = heap[smallest], heap[j]
            j = smallest

        changed = heap != before
        print_heap(
            heap,
            label=f"After sift_down({i}): {'changed' if changed else 'no
            highlight={i},
        )

    print(f"\nFinal heap: {heap}")
```

```
    print("Valid:", end=" ")
    is_min_heap(heap)
    return heap

build_heap_trace([9, 4, 7, 1, 8, 3, 6, 2, 5])
```

Adding `build` to MinHeap

In [ ]:

```python
# Extend MinHeap with a classmethod for O(n) build

class MinHeap(MinHeap):   # inherit everything above

    @classmethod
    def build(cls, iterable):
        """Construct a MinHeap from any iterable in O(n) time.

        Uses Floyd's algorithm: copy all elements then sift-down from
        the last non-leaf toward the root.
        """
        h = cls()
        h._data = list(iterable)
        n = len(h._data)
        # Every index >= n//2 is a leaf; start from the last non-leaf
        for i in range((n - 2) // 2, -1, -1):
            h._sift_down(i)
        return h

# Compare build() vs repeated push()
import time

data = list(range(200_000, 0, -1))   # worst case: reverse sorted

t0 = time.perf_counter()
h_build = MinHeap.build(data)
t_build = time.perf_counter() - t0

t0 = time.perf_counter()
h_push = MinHeap()
```

```
for v in data: h_push.push(v)
t_push = time.perf_counter() - t0

print(f"build()  (Floyd O(n)):   {t_build*1000:.1f} ms")
print(f"n×push() (O(n log n)):  {t_push*1000:.1f} ms")
print(f"Speedup: {t_push/t_build:.1f}×")
print(f"Both valid: {h_build.peek() == h_push.peek()}")
```

Why heaps are cache-friendly

A linked-node tree allocates each node separately → nodes are scattered in memory:

```
Linked tree: root → node_at_0x2a4f → node_at_0x7b11 → node_at_0x1c09 ...
             each pointer dereference → potential cache miss
```

An array-based heap stores all values in one contiguous block:

```
Array heap: [1, 3, 2, 7, 5, 4, 8, 9, 6]  ← lives at one memory address
            parent and children are a few bytes apart
```

Why this matters

- CPU cache lines are 64 bytes; accessing an element brings neighbors along
- Heap operations (sift-up/sift-down) access nearby indices: nearly always in cache
- A linked tree traversal follows pointers to random addresses → cache misses hurt

Result: The array-based heap has the same Big-O but a significantly smaller constant factor.

In [ ]:

```
# Illustrate: which indices does sift_down touch for a heap of size 1,0

def sift_down_path(n, start):
    """Trace indices visited by sift_down starting at 'start' in a heap
    visited = []
    i = start
    while True:
        visited.append(i)
        l, r = 2*i+1, 2*i+2
        if l >= n:
            break
        # go to the side with the smaller hypothetical child
        i = l if (r >= n or l <= r) else r
    return visited

n = 1_000_000
path = sift_down_path(n, 0)

print(f"Heap size: {n:,}")
print(f"Sift-down from root visits {len(path)} nodes (height = log₂(n) ≈
print(f"Indices visited: {path}")
print()
# Show how close together these are in memory (array bytes apart)
for a, b in zip(path, path[1:]):
    diff = abs(b - a)
    print(f"  index {a} → {b}  (distance: {diff} elements = {diff*8} by
```

heapq internals: it's what we just built

CPython's `heapq` module is implemented in Python (with an optional C acceleration). Here's the actual sift-down from the standard library (simplified):

```
def _siftup(heap, pos):
    endpos = len(heap)
    startpos = pos
    newitem = heap[pos]
    # Bubble up the smaller child until hitting a leaf.
    childpos = 2*pos + 1    # leftmost child position
    while childpos < endpos:
        # Set childpos to index of smaller child.
        rightpos = childpos + 1
        if rightpos < endpos and not heap[childpos] < heap[rightpos]:
            childpos = rightpos
        heap[pos] = heap[childpos]
        pos = childpos
        childpos = 2*pos + 1
    heap[pos] = newitem
    _siftdown(heap, startpos, pos)   # one extra sift-up pass
```

This is exactly `_sift_down` from our `MinHeap` — same algorithm, same index formulas.

You now understand CPython's `heapq` from the inside.

In-class Exercise 2 (commit required)

Topic: Build-heap and `MinHeap` in practice

1. **Multiple choice:** Floyd's build-heap visits non-leaf nodes from:

    - A. Root to last non-leaf (top-down)
    - B. Last non-leaf to root (bottom-up)
    - C. Left to right across each level
    - D. In sorted order

2. **Short answer (2–3 sentences):** Explain intuitively why Floyd's build-heap is O(n) even though it calls sift-down (which is O(log n)) on n/2 nodes.

3. **Coding task:** Implement `MinHeap.nsmallest(k)` — a method that returns the k smallest elements from the heap **without destroying it**.

    - Do NOT call `sorted()` or any library sort
    - Hint: you can use a copy of the heap and repeatedly `pop()`, or you can use a different approach
    - Add this method to `MinHeap` (or a subclass) and test with at least 10 elements
    - Verify against `heapq.nsmallest(k, data)`

4. **Commit:**

```
git add lecture/Lecture06_Binary_Heap_Mechanics.ipynb
git commit -m "Lecture 06 exercise 2 work"
```

In [ ]:

```python
# Exercise 2 workspace
import heapq

class MinHeapEx2(MinHeap):

    def nsmallest(self, k):
        """Return the k smallest elements in ascending order.

        Must not destroy or modify the heap.
        """
        # YOUR CODE HERE
        pass

# Test
data = [5, 3, 8, 1, 9, 2, 7, 4, 6, 10, 0, 11]
h = MinHeapEx2.build(data)

k = 4
mine = h.nsmallest(k)
ref  = heapq.nsmallest(k, data)
print(f"MinHeap.nsmallest({k}): {mine}")
print(f"heapq.nsmallest({k}):   {ref}")
print(f"Match: {mine == ref}")

# Verify the original heap is unmodified
print(f"\nHeap still valid after nsmallest: ", end="")
is_min_heap(h._data)
```

In [ ]:
```
# —— SOLUTION HIDDEN ——
# Complete this after the exercise prompt above.
```

Complexity Table

Binary Heap (min-heap, array-based)

Operation	Time	Notes
`push` / insert	**O(log n)**	sift-up: at most one root-to-leaf path
`pop` / extract-min	**O(log n)**	sift-down: same path length
`peek`	**O(1)**	root is always at index 0
`build` (Floyd)	**O(n)**	most nodes are near leaves → little work
`build` (n × push)	**O(n log n)**	naïve approach
Space	**O(n)**	one array, no pointers

Compare to alternatives

Structure	Insert	Extract-min	Build
Unsorted list	O(1)	O(n)	O(1)
Sorted list	O(n)	O(1)	O(n log n)
Binary heap	O(log n)	O(log n)	**O(n)**
BST (balanced)	O(log n)	O(log n)	O(n log n)

Heap beats sorted list on insert; beats unsorted list on extract-min; beats BST on build.

Common Pitfalls

1. Forgetting that sift-down goes to the *smaller* child

Always pick the smaller of left and right before swapping — swapping with the wrong child can violate the heap property for the sibling subtree.

2. Off-by-one in the last non-leaf index

```
# WRONG
for i in range(n // 2, -1, -1): ...    # includes an extra leaf

# CORRECT
for i in range((n - 2) // 2, -1, -1): ...   # last non-leaf
# equivalently: range(n // 2 - 1, -1, -1)   # same result
```

3. Mutating elements after insertion

If you change the priority of an element that's already in the heap, the heap property is violated silently. You must re-heapify (or use the lazy deletion pattern from Lecture 05).

4. Confusing sift-up and sift-down

- **sift-up (insert):** new element goes up until it's ≥ its parent
- **sift-down (pop):** element at root goes down until it's ≤ both children

Key Definitions — full glossary

Term	Definition
Complete binary tree	Every level fully filled except the last, which fills left to right
Heap property	Every parent ≤ both children (min-heap) or ≥ both children (max-heap)
Sift-up / heapify-up	Bubble a new element upward until heap property is restored
Sift-down / heapify-down	Push an element downward (to smaller child) until heap property is restored
Floyd's algorithm	Build a heap in O(n) by sifting-down from last non-leaf to root
Cache-friendly	Operations on nearby memory addresses → fewer cache misses
Index arithmetic	`left=2i+1`, `right=2i+2`, `parent=(i−1)//2` — replaces pointer dereference

Wrap-up

What we built today

- A `MinHeap` class from 30 lines of pure Python: no imports, no pointers
- Step-by-step traces of sift-up and sift-down
- Floyd's O(n) build-heap and the proof sketch for why it works
- Confirmed our implementation matches `heapq` exactly

Key takeaways

1. A heap is **an array** — the tree is an illusion created by index arithmetic
2. Insert and pop are both O(log n) because the tree has height log n
3. Build-heap is O(n) because most nodes are near the bottom and do almost no work
4. Arrays beat linked nodes on real hardware: better cache behavior, smaller memory footprint

Next: Lecture 07 — PQ in Practice: Top-k, Streaming, Event Simulation

- Apply our heap knowledge to real algorithms
- Top-k with a bounded heap, streaming median, Dijkstra's algorithm preview
- Benchmarking: when to use `heapq`, when to reach for `sortedcontainers`